Contents

Fused Causal Attention with NVIDIA cuTile **2**
Implementation, mathematics, testing, benchmarking, demo and viva guide 2
Contents 3

1. Project in one minute **3**
Recorded core results 4
Validated environment snapshot 4
Exact core contract 4

2. Attention mathematics and causal masking **5**
Causal mask illustration 5

3. Why fusion and tiling help **6**
Unfused versus fused 6

4. Online-softmax derivation **7**
Why this works 7

5. Kernel walkthrough **8**
Tiling and work distribution 8
Causal loop bound 8
Main kernel logic 9

6. Precision and input/output contract **10**

7. Source-file map **10**

8. Correctness strategy and evidence **11**
Recorded results 12

9. Benchmark methodology **12**
Frozen workload families 12
Recorded timing protocol 13

10. Results and defensible claims **13**
Long D=128 representative snapshot 13
Longer batched D=128 headline 14
Why 64×64 is the general default 14
Do not claim 14

11. Nsight Compute findings **15**

Interpretation 15

12. Reproducible core demo **15**
A. Run genuine cuTile smoke 16
B. Generate deterministic input 16
C. Run actual cuTile backend 16
D. Verify output against independent oracle 16
E. Run core GPU suite 16
F. One-session benchmark demonstration 16

13. Presentation script **17**

14. Viva questions: fundamentals **18**

15. Viva questions: implementation **19**

16. Viva questions: testing and performance **20**

17. Essential source excerpts **21**

18. Glossary and quick revision **23**
The 30-second answer 23
Five facts to remember 23
Evidence and reference notes 24

Fused Causal Attention with NVIDIA cuTile

Implementation, mathematics, testing, benchmarking, demo and viva guide

Validated target: NVIDIA GeForce RTX 4060, compute capability 8.9 (SM89), native Windows x64
Evidence snapshot: 19 October 2026
Scope: Main completed forward causal self-attention project only.

> **Core idea:** fuse score computation, causal masking, online softmax and value accumulation in a tile-based GPU kernel. Stream key/value tiles and avoid storing the complete N × N score and probability matrices in GPU global memory.

This guide is self-contained for study and presentation. It includes mathematical background, source-file responsibilities, demo commands, recorded evidence, code excerpts and viva questions. It does not require access to the private repository while reading.

Contents

1. Project in one minute
2. Attention mathematics and causal masking
3. Why fusion and tiling help
4. Online-softmax derivation
5. Kernel walkthrough
6. Precision and input/output contract
7. Source-file map
8. Correctness strategy and evidence
9. Benchmark methodology
10. Results and defensible claims
11. Nsight Compute findings
12. Reproducible live demo
13. Presentation script
14. Viva questions: fundamentals
15. Viva questions: implementation
16. Viva questions: testing and performance
17. Essential code excerpts
18. Glossary and quick revision

---

1. Project in one minute

The project implements the forward pass of **causal scaled dot-product self-attention** as a genuine NVIDIA cuTile Python GPU kernel.

An attention operation compares queries (Q) against keys (K) to create scores, normalizes those scores with softmax, and uses them to form a weighted combination of values (V). In causal attention, query position i may use key positions $j \leq i$, but may not read information from future positions $j > i$.

The main performance strategy is to compute the operation in tiles, maintain an online-softmax state, and avoid materializing the full $N \times N$ score and probability matrices in global memory.

Recorded core results

- Genuine cuTile runtime smoke: passed, with exact checking of 4,096 vector-add outputs.
- Core actual-cuTile attention suite: **48 passed, 0 failed**, with `REQUIRE_CUTILE=1`.
- Independent numerical report: **29 legal case/tile configurations**, zero tolerance violations.
- CLI output: finite FP16 output passed an independent NumPy oracle check.
- Benchmarking: three controlled sessions, seven workload families, and up to five legal tile configurations.
- Nsight Compute: two workload configurations, each with basic resource/throughput and scheduler/warp-state profiling.

Validated environment snapshot

Component	Recorded value
GPU	NVIDIA GeForce RTX 4060, SM89 / compute capab
OS / driver	Native Windows x64, build 26300, WDDM; driver 6
Python / NumPy	CPython 3.12.10; NumPy 2.5.4
CuPy / cuTile	CuPy 14.2.0; cuda-tile 1.6.0
Tile compiler packages	TileIRAS, NVCC and NVVM 13.4.92 series
Nsight Compute	CLI 2026.3.1.0

Hardware distinction: these actual-cuTile results belong to the local Windows RTX 4060. The separate Linux A100 runner is a different ordinary-CUDA control environment. Do not mix its results with the RTX 4060 cuTile measurements.

Exact core contract

Inputs Q/K/V are rank-4, contiguous, finite FP16 tensors of shape [B,H,N,D] for the required self-attention path. Batch, head and sequence dimensions must agree; D is one of 32, 64 or 128; and Dv = D. Sequence length must be divisible by both selected tile dimensions. Partial edge tiles are not implemented in the core contract. Output has shape [B,H,N,D] and FP16 dtype.

2. Attention mathematics and causal masking

For batch size B, head count H, sequence length N and head dimension D, each tensor has logical shape [B,H,N,D].

- **Q (Query):** what each token is looking for.
- **K (Key):** used to match a query to a token.
- **V (Value):** the information combined using attention weights.

The scaled score between query i and key j is:

[s_{ij}=\frac{q_i\cdot k_j}{\sqrt D}]

The factor (1/\sqrt D) is part of the standard scaled dot-product attention formula. It helps keep score magnitudes manageable as the feature dimension grows, preventing softmax from becoming excessively concentrated.

For causal attention:

[s_{ij}=\begin{cases} \frac{q_i\cdot k_j}{\sqrt D}, & j\le i\ -\infty, & j>i \end{cases}]

Then:

[p_{ij}=\frac{e^{s_{ij}}}{\sum_{j'=0}^{N-1}e^{s_{ij'}}}, \qquad o_i=\sum_{j=0}^{N-1}p_{ij}v_j]

Because (e^{-\infty}=0), future positions receive zero probability.

Causal mask illustration

Rows are queries and columns are keys:

```
          KEY POSITION
          0   1   2   3
QUERY 0  [OK][X ][X ][X ]
QUERY 1  [OK][OK][X ][X ]
QUERY 2  [OK][OK][OK][X ]
QUERY 3  [OK][OK][OK][OK]
```

The kernel creates absolute query and key positions, then masks positions where `k_pos > q_pos` by replacing their scores with negative infinity. The mask must be applied before softmax; otherwise, future tokens could influence earlier outputs.

A strong causal test changes future K or V values and checks that earlier outputs remain unchanged. This tests the behavioral property directly, rather than only comparing one random output.

---

3. Why fusion and tiling help

A straightforward implementation computes:

1. QK^T.
2. Score scaling.
3. Causal masking.
4. Softmax.
5. Probability × V.

The score and probability matrices each have shape [B,H,N,N]. Their storage therefore grows quadratically with sequence length.

For B=1, H=8, N=2048 and FP32 scores, one score matrix contains:

[1\times8\times2048\times2048]

or 33.6 million FP32 values—128 MiB in binary units. A materialized probability matrix may require a similar amount of storage, before counting input, output and temporary allocations.

Unfused versus fused

Unfused CuPy baseline	Fused cuTile kernel
Separate operations	Fuses score, mask, online softmax and
Materializes full score matrix	Uses local score tiles
Materializes full probability matrix	Streams K/V tiles and updates state
Useful, simple baseline	Optimized kernel under test

The baseline lives in `src/baseline.py`; the actual implementation is in `src/cutile_attention.py`.

Fusion reduces intermediate storage and memory traffic. It **does not** eliminate the $O(N^2)$ query-key interaction count of standard full attention. Also, one fused kernel launch does not mean one block computes the entire sequence: the launch grid distributes work by batch, head and query tile.

4. Online-softmax derivation

The key algorithmic idea is that softmax-weighted values can be accumulated as score tiles arrive. We do not need to store every score until the end.

For each query row, maintain:

State	Meaning	Initial value
m	Maximum score observed so far	$-\infty$
ell	Sum of exponentials shifted by m	0
u	Unnormalized weighted-value numerator	Zero vector

When the next masked score tile S arrives, let (m_t) be its row maximum. Calculate:

[m'=\max(m,m_t),\qquad \alpha=e^{m-m'}]

The new tile's weights are:

[P_t=e^{S-m'}]

The previous accumulated state must be rescaled, then the new contribution added:

[\ell'=\alpha\ell+\sum_jP_{t,j}]

[u'=\alpha u+P_tV_t]

At the end, the output is:

[o=u/\ell]

Why this works

Softmax is unchanged if the same scalar is added to or subtracted from every score in a row. When a new tile has a higher maximum, the old numerator and denominator are both rescaled to the new maximum. Applying the same rescaling to both preserves their ratio. The new tile can then be added on the same exponential scale.

Subtracting the maximum also stabilizes the exponentials: the largest valid exponent becomes exp(0)=1 and the other valid exponents are no greater than one.

The kernel keeps the online maximum, normalization sum, and output accumulator in FP32. An independent NumPy implementation of the tiled recurrence lives in `src/online_reference.py`. The primary GPU oracle remains the separate full-matrix FP32 implementation in `src/reference.py`.

---

5. Kernel walkthrough

The main kernel is built by `_build_kernel()` in `src/cutile_attention.py`.

Tiling and work distribution

- `tile_q` / TQ: query rows handled by a program instance.
- `tile_k` / TK: key/value rows processed during one loop iteration.

For tile 64×64, a query tile has shape [64,D], a K/V tile has 64 rows, and the temporary score tile is [64,64].

The launch grid is:

```
grid = (
    batch,
    heads,
    seq_len // tile.query,
)
```

The three block IDs identify batch, head and query tile. One program instance loads its query tile once, then loops over the K/V tiles relevant to those queries.

Causal loop bound

The kernel computes:

```
num_k_tiles_for_query = ct.cdiv(
    (query_block + 1) * TQ,
```

```
    TK,
)
```

This skips complete K/V tiles that are wholly beyond the last query position in the query tile. The element-wise causal mask is still required because a partly valid tile may contain future keys for some query rows.

Main kernel logic

```
score_acc = ct.zeros((TQ, TK), dtype=ct.float32)
scores = ct.mma(q, ct.transpose(k), score_acc)
scores = scores * SCALE

future = k_pos > q_pos
scores = ct.where(future, float("-inf"), scores)

tile_max = ct.max(scores, axis=1, keepdims=True)
new_max = ct.maximum(running_max, tile_max)
rescale = ct.exp(running_max - new_max)
probabilities = ct.exp(scores - new_max)

new_norm = rescale * running_norm + ct.sum(
    probabilities, axis=1, keepdims=True
)
probabilities16 = probabilities.astype(ct.float16)
previous_out = rescale * running_out
running_out = ct.mma(
    probabilities16, v, previous_out
)
running_norm = new_norm
running_max = new_max
```

After the loop, the kernel calculates `output = running_out / running_norm`, reshapes the tile, casts to output dtype, and stores it. The Python wrapper synchronizes the stream before copying the result back to a NumPy array. The low-level launch function can work on arrays already resident on the GPU, which is useful for benchmarking.

---

6. Precision and input/output contract

Stage	Project precision / rule
Q/K/V storage	FP16
QK^T accumulator	FP32
Score scaling and online-softmax state	FP32
Running normalization sum ell	FP32
Probability operands for P×V	Converted to FP16
P×V running accumulator	FP32
Output	FP16

Describe this as **mixed precision with FP32 accumulation**, not "everything is computed in FP32." Probabilities are converted to FP16 operands for P×V, but the running output accumulator remains FP32. That rounding trade-off is one reason the independent accuracy gate is important.

The accepted core path requires:

- Q/K/V rank 4, FP16, contiguous and finite.
- Matching batch, head and sequence dimensions.
- Matching Q and K feature dimensions.
- D ∈ {32,64,128}, with Dv = D.
- Sequence length divisible by tile_q and tile_k.
- Finite FP16 output of the expected shape.

The fixed mixed tolerance is:

[|O_{actual}-O_{reference}|\le 0.02+0.02|O_{reference}|]

The absolute term matters near zero; the relative term scales with the reference magnitude. It is a fixed acceptance rule, not something to relax just to make a failing optimization pass.

---

7. Source-file map

File	Responsibility
`src/cutile_attention.py`	Actual cuTile kern

File	Responsibility
src/reference.py	Independent full-r
src/online_reference.py	Independent tiled
src/contract.py	Shared input and
src/baseline.py	Deliberately unfus
src/attention_io.py	NPZ loading/valida
run_attention.py	Course-facing CLI
scripts/cutile_smoke.py	Genuine cuTile co
scripts/generate_inputs.py	Deterministic test
scripts/verify_cli_output.py	Verifies saved out
tests/test_gpu_correctness.py	Main correctness s
tests/test_gpu_edge_cases.py	Causal invariants,
tests/cases.py	Frozen workload s
benchmarks/benchmark_attention.py	CUDA-event timin
benchmarks/profile_fused.py	Prepares a warme
Report.md	Experiment narrat
docs/CUTILE_RTX4060_RESULTS_2026-10-19.md	Accuracy, benchm
docs/CUTILE_NCU_PROFILE_ANALYSIS_2026-10-19.md	Profiler metrics an

Two distinctions to remember: the full-matrix NumPy oracle is not the optimized kernel; and ordinary-CUDA control implementations are not proof that cuTile was executed. The cuTile smoke and actual attention run establish the real cuTile path.

---

8. Correctness strategy and evidence

Correctness was established in layers because runtime bring-up, mathematical correctness, CLI behavior and performance are different claims.

1. **Runtime smoke:** compiles, launches and synchronizes a genuine `cuda.tile` vector-add kernel; checks 4,096 outputs exactly. This proves the runtime works, not attention correctness by itself.
2. **Independent oracle:** full-matrix FP32 NumPy implementation computes expected attention output without calling the optimized kernel.
3. **GPU regression:** checks supported dimensions and workloads, output shape/dtype, numerical tolerance, finite values

and invalid inputs.

4. **Causal invariants:** changing future K or V must not affect earlier outputs.
5. **CLI gate:** saved output is checked for shape, FP16 dtype, finite values and agreement with the oracle.

Recorded results

Gate	Preserved core result
Genuine cuTile smoke	PASS; exact 4,096 element c
Core GPU test suite	48 passed, 0 failed, REQUIRE
Independent numerical matrix	29 legal case/tile configurati
Maximum absolute error	0.0009765625
Largest per-configuration P99 absolute error	0.00048828125
Largest per-configuration RMSE	0.000086860741
Maximum error / allowed tolerance	0.0243665, about 2.44% of t
CLI/reference output	PASS; expected FP16 shape a

The 2.44% figure is the fraction of the tolerance envelope consumed by the worst element. It does not mean the output was 2.44% wrong.

---

9. Benchmark methodology

A useful GPU benchmark separates first-call/JIT-associated latency from steady-state execution, accounts for asynchronous launches, measures repeated samples and names the baseline.

Frozen workload families

Case	B	H	N	D
short_d32	1	1	64	32
short_d64	1	1	64	64
short_multihead_d64	1	4	128	64
medium_d64	1	8	256	64
medium_d128	1	8	512	128

Case	B	H	N	D
long_d128	1	16	1024	128
longer_batched_d128	2	8	2048	128

Tile candidates, where legal: 32×64, 64×64, 64×128, 128×64 and 128×128. Some are excluded from some workloads when the sequence length is not divisible by both tile dimensions. This is why the accuracy report contains 29 legal case/tile configurations rather than 35 possible pairs.

Recorded timing protocol

- 10 warmup iterations.
- 30 CUDA-event samples per configuration/session.
- Five launches averaged inside each event sample.
- Three independent sessions, with tile order rotated.
- Median and P90 latency.
- Baseline measurements before and after the tile sweep; average reported and drift retained.
- First-call wall time recorded separately; it may include a compiler-cache hit and is not necessarily pure compilation time.

The baseline is the repository's deliberately unfused FP32 CuPy implementation. Both implementations use arrays already resident on the GPU; fused timing includes output allocation and the device launch but excludes host-to-device input copies. It is a kernel-path comparison, not end-to-end application latency including file I/O and transfers.

Speedup = unfused baseline median / fused cuTile median.

---

10. Results and defensible claims

Long D=128 representative snapshot

Implementation	Median latency	Speedup
Unfused FP32 CuPy baseline	4.414310 ms	1.00×
Fused cuTile, 64×64	0.267059 ms	16.53×
Fused cuTile, 64×128	0.272794 ms	16.18×

The long D=128 result had 64×64 and 64×128 practically tied in the root summary: 0.273408 ms versus 0.273203 ms, about 0.08% apart. The report therefore retains 64×64 as the general default.

Longer batched D=128 headline

The root report headline gives 0.919347 ms median, 0.924795 ms P90 and 18.62×. The detailed results table gives a nearby snapshot at 0.923034 ms median, 0.937656 ms P90 and 18.54×. These are different preserved benchmark snapshots. The safe common summary is approximately **0.92 ms and 18.5–18.6× versus the unfused FP32 CuPy baseline**. When quoting exact numbers, keep the median/P90/speedup from the same snapshot.

Why 64×64 is the general default

- The long D=128 case shows 64×64 and 64×128 essentially tied.
- 64×128 was slightly faster for the longer batched D=128 workload.
- The larger tile has a greater shared-memory footprint in the profiled longer workload.
- Short workloads showed significant Windows/WDDM timing noise, so their speedups are not the headline claim.

Do not claim

- 18.6× faster than FlashAttention: that comparison was not performed.
- Output-only throughput is total DRAM bandwidth.
- Profiler-instrumented duration equals ordinary benchmark latency.
- The largest tile is always fastest.
- An Nsight heuristic estimate is a measured optimization result.

11. Nsight Compute findings

Nsight Compute profiles a GPU kernel and reports latency, registers, shared memory, occupancy, throughput and warp-scheduling statistics. The project profiler warms a configuration and then launches a single steady-state kernel in the profiling region.

Metric	Medium D=128, 64×64	Longer b
Tensor shape	B=1, H=8, N=512	
NCU kernel duration	52.90 µs	
Registers per thread	231	
Static shared memory/block	65.54 KiB	
Achieved occupancy	8.35%	
SM throughput vs peak	18.81%	
DRAM throughput vs peak	22.88%	
Scheduler cycles with no eligible warp	85.83%	

Interpretation

Shared memory is the clearest per-block residency constraint in these profiles. The footprint is around 65.5–81.9 KiB per block, and achieved occupancy is around 8.3%. Register use is substantial too. Many scheduler cycles had no eligible warp ready to issue in these samples.

These observations guide hypotheses for further investigation, but they do not fully attribute every stall. Nsight's rule-based speedup estimates are heuristics, not measured optimization gains. The more detailed PC-sampling metric was unavailable in the preserved exports.

The sensible decision was to retain the validated tile choices and investigate resource footprint or instruction scheduling only through a new variant that passes correctness and controlled benchmarks.

---

12. Reproducible core demo

Commands assume PowerShell at the repository root on the Windows RTX 4060 host and the project-local environment already configured.

A. Run genuine cuTile smoke

```
$py = ".\.venv-cutile-windows\Scripts\python.exe"
& $py scripts\cutile_smoke.py
```

Expected markers: `OUTPUT_CHECK=PASS` and `CUTILE_SMOKE=PASS`. This proves genuine cuTile compilation/execution, not attention correctness by itself.

B. Generate deterministic input

```
New-Item -ItemType Directory -Force .tmp\demo | Out-Null
& $py scripts\generate_inputs.py --output .tmp\demo\input.npz -
```

C. Run actual cuTile backend

```
& $py run_attention.py --input .tmp\demo\input.npz --output .tm
```

The CLI prints the backend, Q/K/V shapes and output shape/dtype, then writes an FP16 NPY array. N=64 is divisible by both 64-sized tiles and D=32 is supported.

D. Verify output against independent oracle

```
& $py scripts\verify_cli_output.py --input .tmp\demo\input.npz
```

Expected marker: `CLI_REFERENCE_CHECK=PASS`.

E. Run core GPU suite

```
$env:REQUIRE_CUTILE = "1"
& $py -m pytest -q tests\test_gpu_correctness.py tests\test_gpu
$testExit = $LASTEXITCODE
Remove-Item Env:REQUIRE_CUTILE
if ($testExit -ne 0) { throw "Core GPU tests failed" }
```

The recorded core run passed 48 tests. A live rerun can depend on the current software/driver/GPU environment, so report the result that actually appears.

F. One-session benchmark demonstration

```
& $py benchmarks\benchmark_attention.py --case longer_batched_d
```

This creates JSON and CSV results. One invocation is one session; the published project summary is based on three sessions with rotated tile order.

---

13. Presentation script

Use this as an 8–10 minute presentation outline and adapt it to your speaking style.

Opening: “My project implements causal scaled dot-product self-attention as a fused NVIDIA cuTile Python GPU kernel. The goal is to preserve the attention result while avoiding the full score and probability matrices in GPU global memory.”

Problem: “The standard computation forms QK transpose, scales it, applies the causal mask, performs softmax and multiplies by V. Materializing both N-by-N intermediates becomes expensive as sequence length grows.”

Algorithm: “The kernel works on query and key/value tiles. It keeps a running maximum, normalization sum and weighted-value numerator. When a new tile arrives, the previous state is rescaled to the new maximum and the tile's contribution is added.”

Causality: “For query position i, only key positions $j \leq i$ are permitted. Future scores become negative infinity before softmax, and entire key/value tiles beyond the current query tile's causal range are skipped.”

Precision: “Q/K/V are FP16. The score accumulator and online-softmax state are FP32. Probabilities are converted to FP16 operands for the second matrix multiplication while the running output accumulator stays FP32. The final output is FP16.”

Correctness: “We compare against an independent full-matrix FP32 NumPy oracle. The preserved core suite passed 48 tests and the separate numerical matrix passed 29 legal case/tile configurations without tolerance violations.”

Performance: “We use CUDA events, warmups, 30 samples, five launches per sample, rotated tile order and three sessions. The measured speedup is versus our deliberately unfused FP32 CuPy baseline, not FlashAttention or vendor SDPA.”

Conclusion: “The core result is a real cuTile fused attention implementation, validated against independent mathematics, measured systematically, and profiled to understand its resource behavior.”

---

14. Viva questions: fundamentals

1. What problem does the project solve?
It implements the forward pass of causal scaled dot-product self-attention as a fused GPU kernel and avoids materializing the full score/probability matrices in global memory.

2. What are Q, K and V?
Queries express what a token is looking for; keys are matched against queries; values are the information combined using the resulting attention weights.

3. What is causal attention?
Query position i can use key positions $j \leq i$. Future positions $j > i$ are masked.

4. Why scale by sqrt(D)?
It helps keep the dot-product score magnitudes manageable as the feature dimension increases.

5. Why apply the mask before softmax?
A negative-infinity score exponentiates to zero, so future keys receive zero probability.

6. What does softmax do?
It converts score values into nonnegative weights that sum to one for each query row.

7. What does fused mean?
Score calculation, scaling, causal masking, online softmax and weighted-value accumulation occur inside one cuTile kernel program instead of separate materializing operations.

8. Does fusion remove $O(N^2)$ work?
No. Standard full attention still computes a quadratic number of query-key interactions. Fusion reduces intermediate storage and memory traffic.

9. What is NVIDIA cuTile?
A tile-based Python GPU programming model used to express and

compile the project's actual GPU kernel.

10. What is a tile?
A block of tensor elements processed as a unit. tile_q controls query rows per program; tile_k controls K/V rows per loop iteration.

---

15. Viva questions: implementation

11. Does one GPU block process the whole sequence?
No. The grid is indexed by batch, head and query tile. Each program owns a query tile and loops through relevant K/V tiles.

12. What are m, ell and u?
m is the running maximum; ell is the sum of shifted exponentials; u is the unnormalized weighted-value numerator. The final output is u/ell.

13. Why does online softmax work?
When a tile raises the maximum, the old numerator and denominator are rescaled by exp(old_max − new_max), preserving their ratio before the new tile contributions are added.

14. Why both a causal loop bound and an element-wise mask?
The bound skips whole K/V tiles that are entirely future. The mask handles future positions inside any tile that is only partly valid.

15. What does tile_q control?
The number of query rows handled by one kernel program instance.

16. What does tile_k control?
The number of K/V rows processed in each loop iteration.

17. What input format does the CLI accept?
An NPZ file with Q, K and V arrays satisfying the shape, dtype, contiguity and finiteness contract.

18. What output does it write?
A single NPY output array with shape [B,H,N,D] and FP16 dtype for the core contract.

19. Why reject non-divisible sequence lengths?
Partial edge tiles are outside the core contract. Clear rejection is safer than silently producing incomplete work.

20. Why use a separate online NumPy reference?
It validates the online recurrence independently, helping separate mathematical errors from GPU DSL, indexing, launch or compiler errors. The primary output oracle is still the full-matrix NumPy implementation.

---

16. Viva questions: testing and performance

21. What precision does the kernel use?
FP16 inputs and output; FP32 score accumulation and online-softmax state; FP16 probability operands for P×V while its running accumulator remains FP32.

22. How is correctness measured?
Against an independent full-matrix FP32 NumPy oracle, with mixed tolerance abs(error) ≤ 0.02 + 0.02×abs(reference), plus shape/dtype/finite checks and causal invariants.

23. What is the difference between 48 tests and 29 configurations?
48 is the recorded count of core GPU tests. 29 is the count of legal case/tile combinations in the separate numerical report. They are different evidence.

24. What is the performance baseline?
The intentionally unfused FP32 CuPy implementation that materializes the score and probability matrices.

25. Can you claim 18.6× over FlashAttention?
No. The recorded speedup is against the unfused CuPy baseline only.

26. Why CUDA events?
GPU launches are asynchronous. Events measure elapsed time on the GPU stream rather than only host-side launch overhead.

27. Why warmups and repeated samples?
They reduce first-use effects and sampling noise. Multiple sessions with rotated tile order reduce reliance on one measurement or ordering.

28. Why measure the baseline before and after the tile sweep?
It reveals timing drift during the session; significant drift is a warning that environmental noise may affect comparisons.

29. What did Nsight Compute reveal?
In two sampled configurations it reported roughly 8.3% achieved occupancy, 231–255 registers per thread, 65.5–81.9 KiB static shared memory per block and many scheduler cycles with no eligible warp.

30. Does low occupancy prove the kernel is bad?
No. Occupancy is one metric. It must be interpreted alongside latency, throughput, resource usage and useful work.

31. Why is the largest tile not always fastest?
Larger tiles can improve reuse but require more shared memory and registers. The best setting depends on workload.

32. What does the speedup result prove?
It demonstrates a speed improvement versus the specified unfused CuPy baseline under the recorded benchmark protocol, not superiority over every attention implementation or end-to-end application.

33. Why use RTX 4060 instead of A100 for these cuTile numbers?
The validated actual-cuTile runs were on the Windows RTX 4060. The Linux A100 was a separate ordinary-CUDA control environment; its results should not be mixed with the cuTile numbers.

34. What would you investigate next?
Use profiler evidence to form a resource/scheduling hypothesis, implement a controlled variant, rerun correctness, and accept it only if independent benchmark sessions show a repeatable gain.

---

17. Essential source excerpts

The main online-softmax update in `src/cutile_attention.py` follows this structure:

```
tile_max = ct.max(scores, axis=1, keepdims=True)
new_max = ct.maximum(running_max, tile_max)
rescale = ct.exp(running_max - new_max)
probabilities = ct.exp(scores - new_max)
```

```
new_norm = rescale * running_norm + ct.sum(
    probabilities, axis=1, keepdims=True
)
probabilities16 = probabilities.astype(ct.float16)
previous_out = rescale * running_out
running_out = ct.mma(
    probabilities16, v, previous_out
)
running_norm = new_norm
running_max = new_max
```

This excerpt begins after score scaling and causal masking. The probabilities for masked future positions are zero after exponentiation. The running state remains FP32.

The launch path allocates the output, builds the grid and launches on the current stream:

```
out_gpu = cp.empty(
    (batch, heads, seq_len, value_dim),
    dtype=cp.float16,
)
grid = (batch, heads, seq_len // tile.query)
scale = 1.0 / np.sqrt(head_dim)
ct.launch(
    cp.cuda.get_current_stream(),
    grid,
    kernel,
    kernel_args,
)
```

The independent NumPy oracle uses the straightforward full-matrix path:

```
scores = np.matmul(q32, np.swapaxes(k32, -1, -2))
scores *= np.float32(1.0 / math.sqrt(q.shape[-1]))
scores = np.where(future_mask, -np.inf, scores)
row_max = np.max(scores, axis=-1, keepdims=True)
weights = np.exp(scores - row_max)
probs = weights / np.sum(weights, axis=-1, keepdims=True)
out = np.matmul(probs, v32)
result = out.astype(np.float16)
```

The complete source adds input validation, finite-output checks and

error handling. Run the actual project files for demos rather than copying only these abbreviated snippets.

---

18. Glossary and quick revision

Term	Meaning
Attention	Weighted combination of value vectors using query-ke
Causal mask	Prevents query i from using future keys j > i
cuTile	NVIDIA tile-based Python GPU programming model us
Fusion	Combining stages to avoid materializing intermediate
Global memory	GPU device memory accessible to kernels
Online softmax	Incremental softmax accumulation across score tiles
FP16	16-bit floating point for storage/output
FP32 accumulation	32-bit floating point for sensitive accumulators
CUDA event	GPU stream event used to measure elapsed device tir
Median / P90	50th and 90th percentile latency
Occupancy	Active warps relative to hardware-supported capacity
Nsight Compute	NVIDIA profiler for kernel timing, resources and sched
Independent oracle	Separately written reference used to check optimized
Baseline drift	Change in baseline timing before versus after a bench

The 30-second answer

"I implemented causal scaled dot-product self-attention in a genuine cuTile Python fused GPU kernel. It computes QK scores tile by tile, applies the causal mask, updates an FP32 online-softmax state, and accumulates weighted values without storing the full score/probability matrices in GPU global memory. The recorded core suite passed 48 GPU tests and 29 legal numerical configurations. Benchmarks used controlled CUDA-event timing against an unfused FP32 CuPy baseline, and Nsight Compute showed substantial shared-memory/register usage and low occupancy in the sampled configurations."

Five facts to remember

1. **Math:** softmax(QK^T / sqrt(D) + causal mask) V.

2. **Optimization:** tiled incremental calculation; no full N×N score/probability matrices in global memory.
3. **Precision:** FP16 inputs/output; FP32 accumulators and online state; FP16 probability operands for P×V.
4. **Evidence:** 48 GPU tests, 29 legal numerical configurations, 3 benchmark sessions and 2 profiled workload configurations.
5. **Claim boundary:** about 18.5–18.6× for the longer batched case versus the unfused FP32 CuPy baseline, not versus FlashAttention.

Evidence and reference notes

The figures above are the preserved successful core-validation snapshot dated 19 October 2026. Later repository changes were still awaiting native-GPU regression at guide preparation time; these values should not be represented as revalidation of every later commit.

The two retained longer-batched timing snapshots differ slightly: root report 0.919347 ms median / 0.924795 ms P90 / 18.62×; detailed results table 0.923034 ms / 0.937656 ms / 18.54×. The robust common conclusion is about 0.92 ms and 18.5–18.6× against the unfused baseline.

Principal sources in the project: `Report.md`; `docs/COURSE_PROJECT_REPORT_10-19.md`; `docs/CUTILE_RTX4060_RESULTS_2026-10-19.md`; `docs/CUTILE_NCU_PROFILE_ANALYSIS_2026-10-19.md`. Background documentation: NVIDIA cuTile Python Quickstart, CuPy installation guide, NVIDIA Nsight Compute documentation, and Vaswani et al., *Attention Is All You Need* (2017).